

SET / UNORDERED SET

```
set = sorted,  $O(\log n)$ . unordered_set = no order,  $O(1)$  average (faster).
set<int> s;
s.insert(x); // adds x (ignored if already present)
auto res = s.insert(x); // res.second = true if x was newly inserted
s.count(x); // 1 if exists, 0 if not
s.erase(x); // remove by value
s.erase(s.find(x)); // remove by iterator
*s.begin(); // smallest element
*prev(s.end()); // largest element
s.lower_bound(x); // iterator to first element  $\geq x$ 
s.upper_bound(x); // iterator to first element  $> x$ 
s.size(); s.empty(); s.clear();
```

```
unordered_set<int> us; // same syntax, faster, unordered
// Check if array contains all numbers 1..n exactly once
set<int> s(a.begin(), a.end());
bool valid = (s.size() == n) &&
             (*s.begin() == 1) &&
             (*prev(s.end()) == n);
```

MULTISET

Like set, but allows duplicate values while staying sorted.

```
multiset<int> ms;
ms.insert(x); // allows duplicates
ms.count(x); // how many times x appears
ms.find(x) != ms.end(); // existence check
ms.erase(ms.find(x)); // erase ONE occurrence
ms.erase(x); // erases ALL occurrences of x
*ms.begin(); // smallest element
*prev(ms.end()); // largest element
ms.lower_bound(x); // first element  $\geq x$ 
ms.upper_bound(x); // first element  $> x$ 
ms.size(); ms.empty(); ms.clear();
// Maintain a running sorted multiset and always get current min/max
multiset<int> ms;
ms.insert(5); ms.insert(3); ms.insert(5);
int mn = *ms.begin(); // 3
int mx = *prev(ms.end()); // 5
ms.erase(ms.find(5)); // removes just ONE of the two 5's
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template<typename T>
using ordered_set =
tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
template<typename T>
using ordered_multiset =
tree<T, null_type, less_equal<T>, rb_tree_tag, tree_order_statistics_node_update>;
```